[NAME]: GAYATHRI S

[STUDENT_ID]: (26007F_012)

[PROJECT TITLE]: LEXICAL ANALYZER

[DATE]:18/07/2026

[DESCRIPTION]

[Introduction]

Lexical Analyser is the first phase of a compiler, also called as Scanner. It reads the source program character by character and converts it into meaningful tokens such as keywords, identifiers, operators and constants by removing whitespace and comments. It acts as an interface between the source code and the parser and creates a symbol table for further processing.

[Objectives]

=> The main objective of the Lexical Analyser is to read the source code and break it into a sequence of tokens for the syntax analyser.
=> It removes unwanted whitespace and comments, identifies keywords, identifiers, operators and constants, and creates a symbol table to store identifier information.

[Working principle]

The Lexical Analyser works by reading the source program character by character from left to right. It groups the characters into meaningful tokens like keywords, identifiers and operators using pattern matching. It skips whitespace and comments, checks for invalid characters, and stores identifiers in the symbol table. Finally, it sends the generated tokens one by one to the Syntax Analyser for further processing.

[File Structure]

1.main.c

2.text file ->. to write a program

[Advantages of lexical analyzer]

1.Converts source code into tokens.
2.Removes whitespace and comments.
3.Detects lexical errors early.
4.Creates a symbol table for compiler.
5.Makes parsing easy and fast.

[Conclusion]

In conclusion, the Lexical Analyser is an essential first phase of a compiler that simplifies the compilation process by converting source code into tokens. It removes unwanted characters and detects errors early, thereby improving the efficiency and accuracy of the compiler.